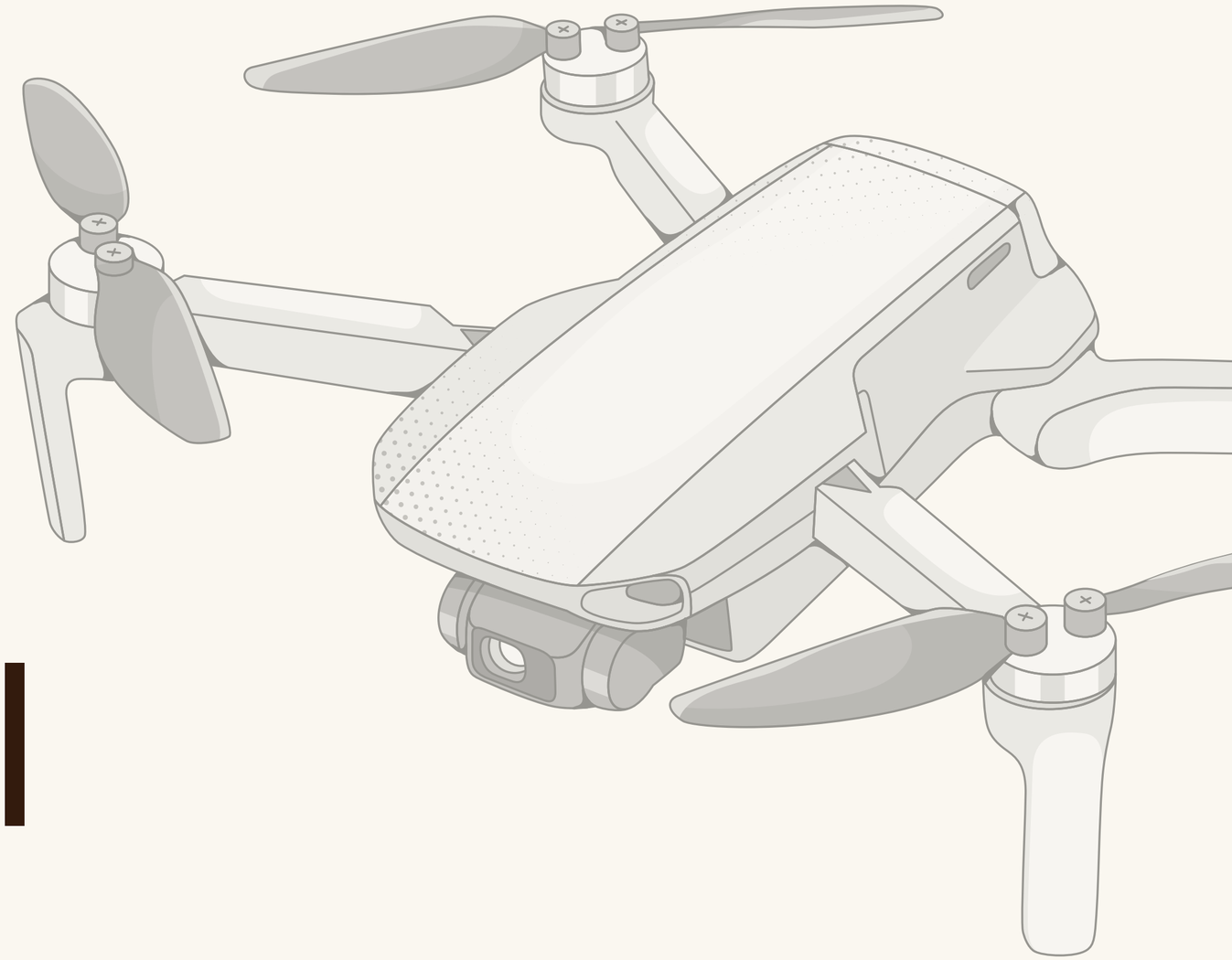
RRC SUMMER SCHOOL

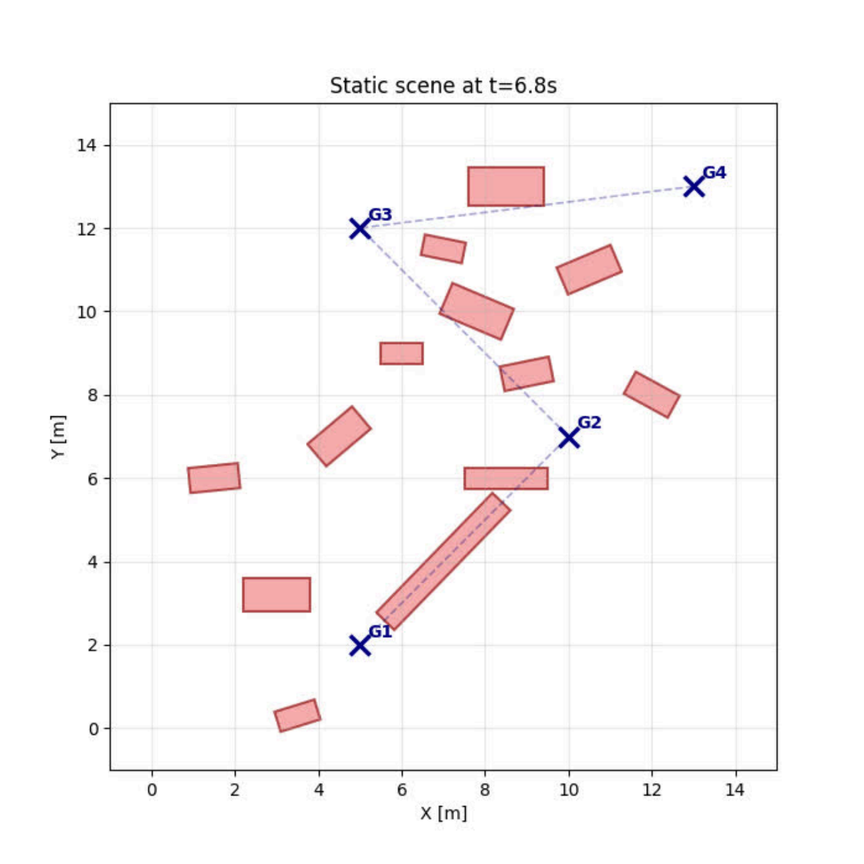
Motion Planning - III

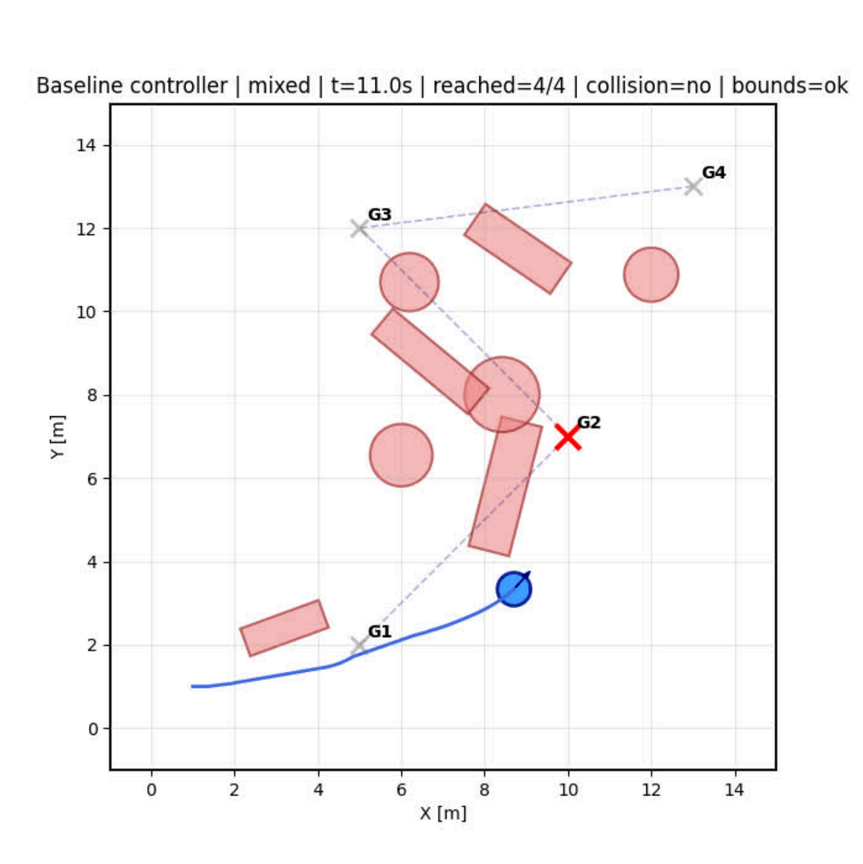


MATHAI MATHEW

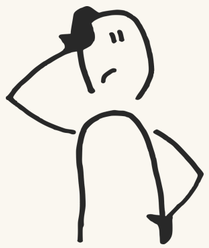
BASICS

“All the world’s a stage”





The static world assumption



Problem

- Our robot can move, so can other obstacles like humans, and other robots or objects.
- Isn’t **assuming a static world** dangerous?

Solution

If obstacles have velocity, shouldn't our **collision checking happen in velocity space**, not just position space?

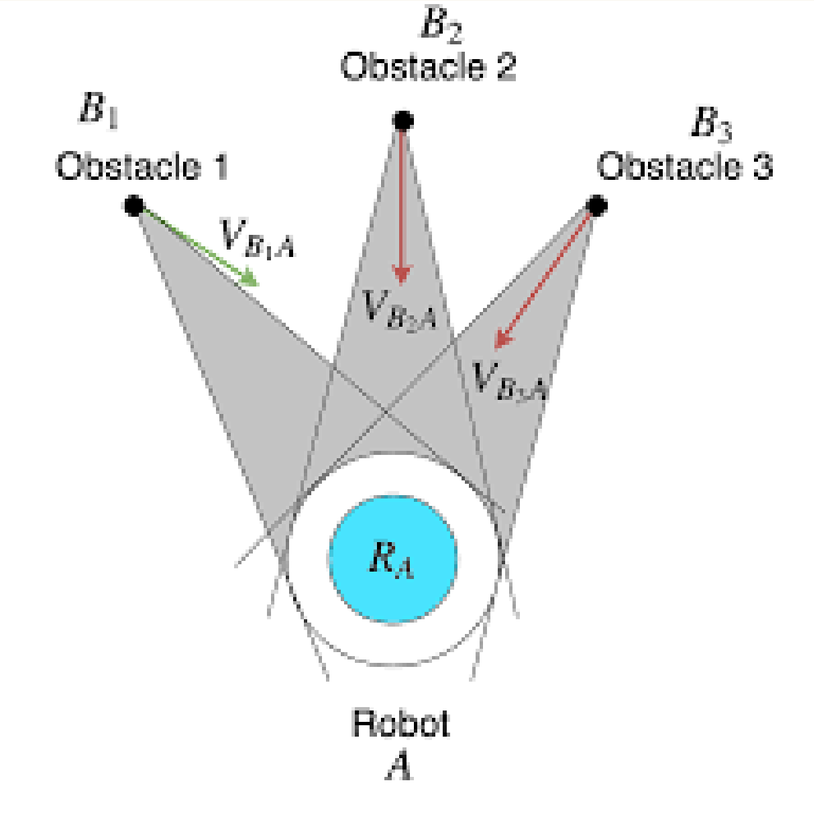
What is a Velocity Obstacle?

$$\mathrm{VO}_{A|B} = \mathbf{v}_A \mid \exists t > 0 : \mathbf{p}_A + \mathbf{v}_A t \in \mathcal{D}(\mathbf{p}_B + \mathbf{v}_B t, r_A + r_B)$$

$\mathbf{p}_A + \mathbf{v}_A t$ - where the robot will be

$\mathbf{p}_B + \mathbf{v}_B t$ - where the obstacle will be

$r_A + r_B$ - combined radius



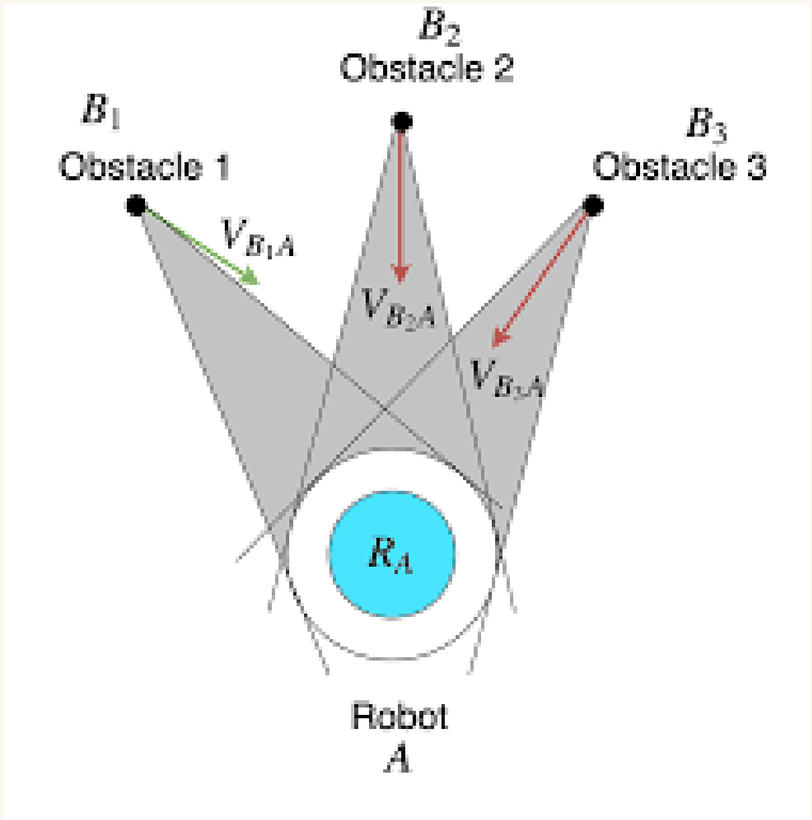
The geometric construction

$$\mathrm{VO}_{A|B} = \mathbf{v}_A \mid \exists t > 0 : \mathbf{p}_A + \mathbf{v}_A t \in \mathcal{D}(\mathbf{p}_B + \mathbf{v}_B t, r_A + r_B)$$

$\mathbf{v}_B$ - the apex of the cone

$\mathbf{p}_A - \mathbf{p}_B$ - the direction the cone points to

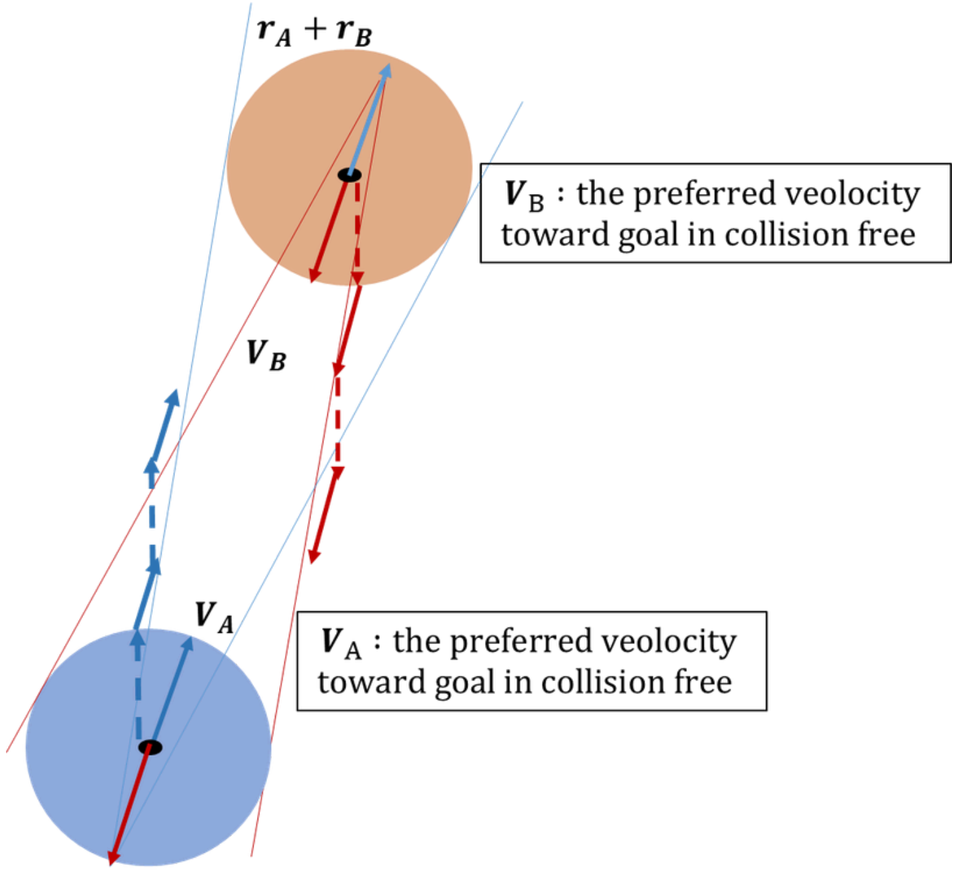
$\sin\alpha = \dfrac{r_A + r_B}{|\mathbf{p}_B - \mathbf{p}_A|}$ - half angle of the cone



Reciprocal Velocity Obstacles: multi-agent

$$\mathrm{RVO}_{A|B} = \mathbf{v}_A \mid 2\mathbf{v}_A - \mathbf{v}_B \in \mathrm{VO}_{A|B}$$

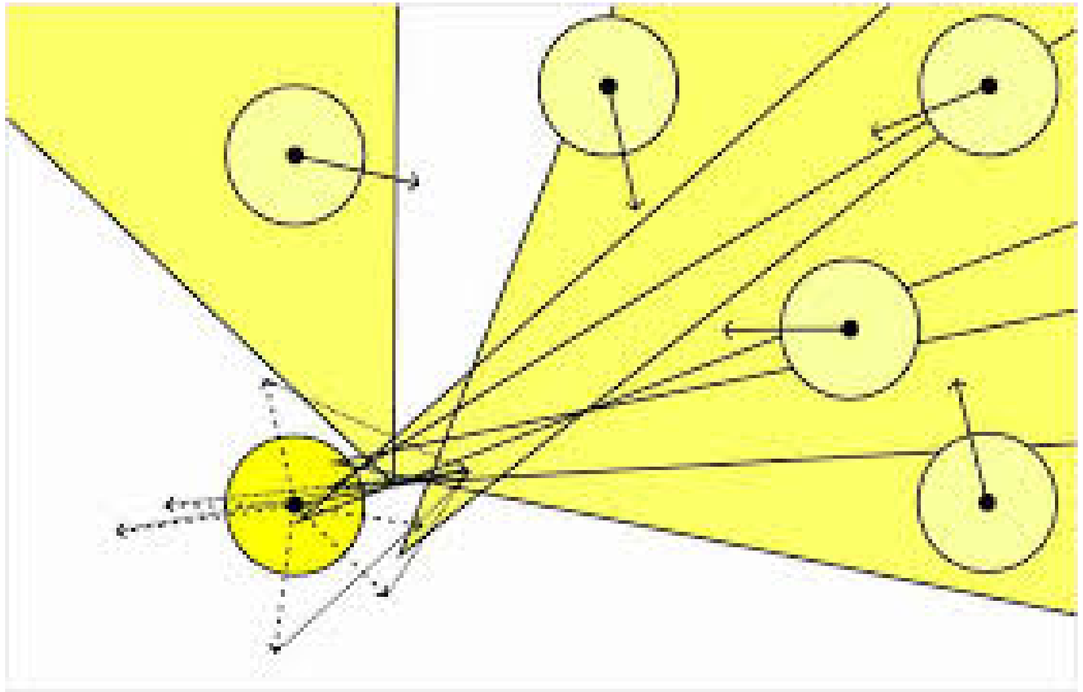
- Robot A acts as if it will make half the velocity change needed for avoidance, assuming B will make the other half.
- robots don't need to communicate or coordinate explicitly. If both use RVO, cooperative avoidance emerges automatically.



Reciprocal Velocity Obstacles: multi-agent

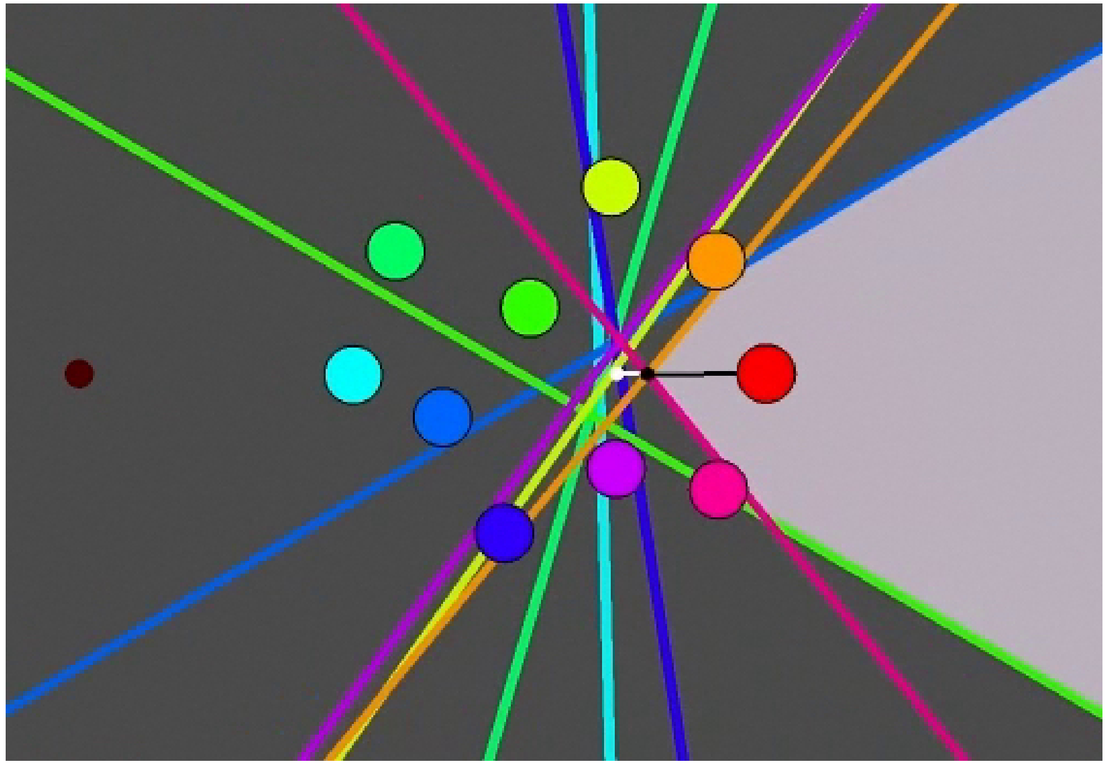
$$\mathrm{RVO}_{A|B} = \mathbf{v}_A \mid 2\mathbf{v}_A - \mathbf{v}_B \in \mathrm{VO}_{A|B}$$

- In a simulation environment with a large number of obstacles, this leads to a few problems such as the whole space being in the region of a velocity obstacle and thus non-navigable.
- For this we can use methods like ORCA which can more efficiently give us the optimal solution



ORCA

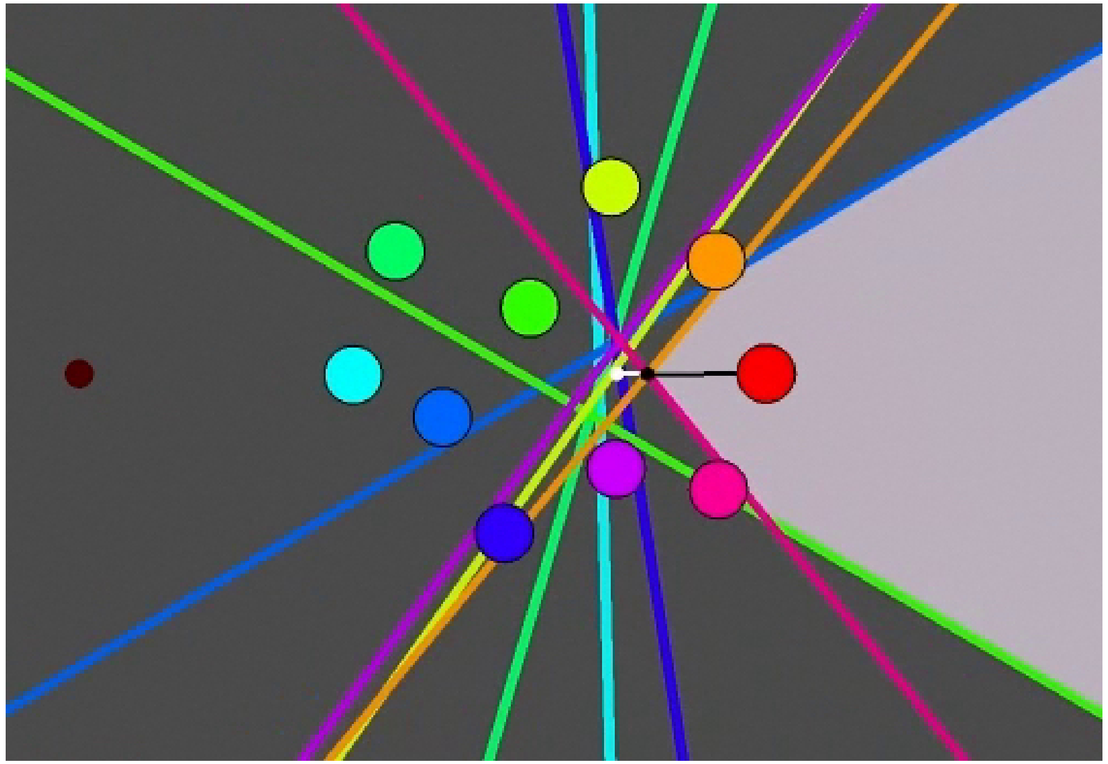
- ORCA uses the concept of half planes instead of inefficient cones. the benifit of this is that this is a convex problem, unlike in the cone case, and so we are able to easily get the safe velocity we require, for all objects.



Putting it all together

$$\mathbf{v}_{\text{sel}} = \arg\min \mathbf{v} \in \mathcal{V}_{\text{safe}} |\mathbf{v} - \mathbf{v}_{\text{pref}}|$$

- Finally after getting all the safe and unsafe velocity values from our velocity obstacle/ORCA, we can then get the ideal velocity we need by choosing the one that is closest to the ideal velocity assuming there are no obstacles.



Time scaling: separating shape from speed

A path is a geometric curve parameterized by arc length s:

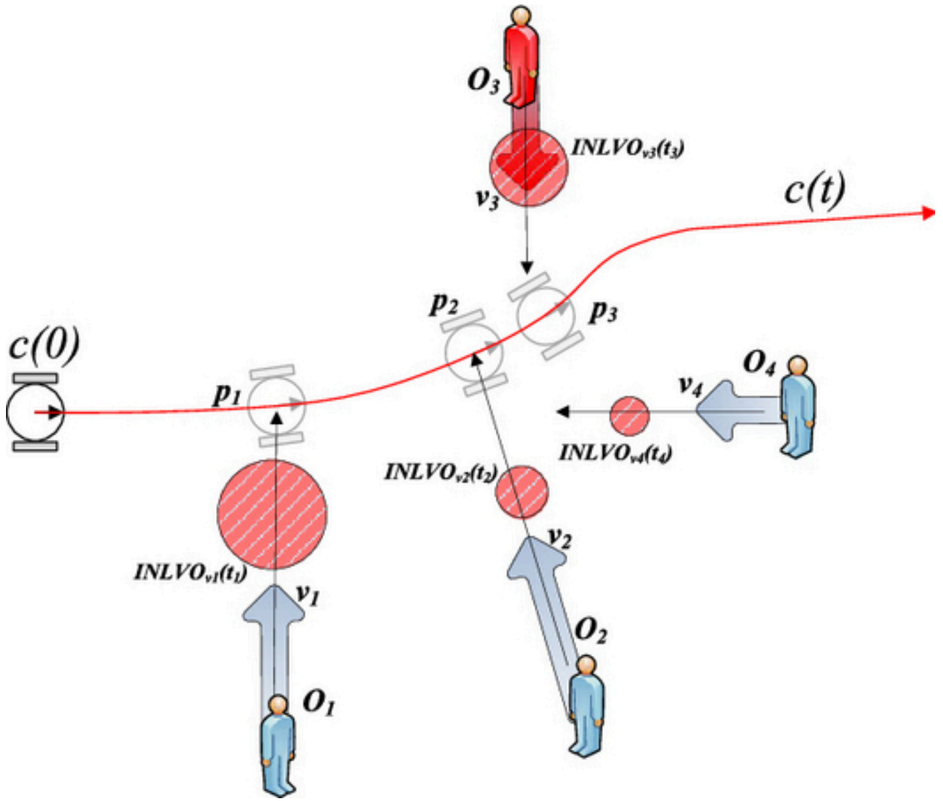
$$\mathbf{q}(s), \quad s \in [0, S]$$

A time scaling is a function that maps time to arc length:

$$c(t), \quad t \in [0, T]$$

The trajectory is then:

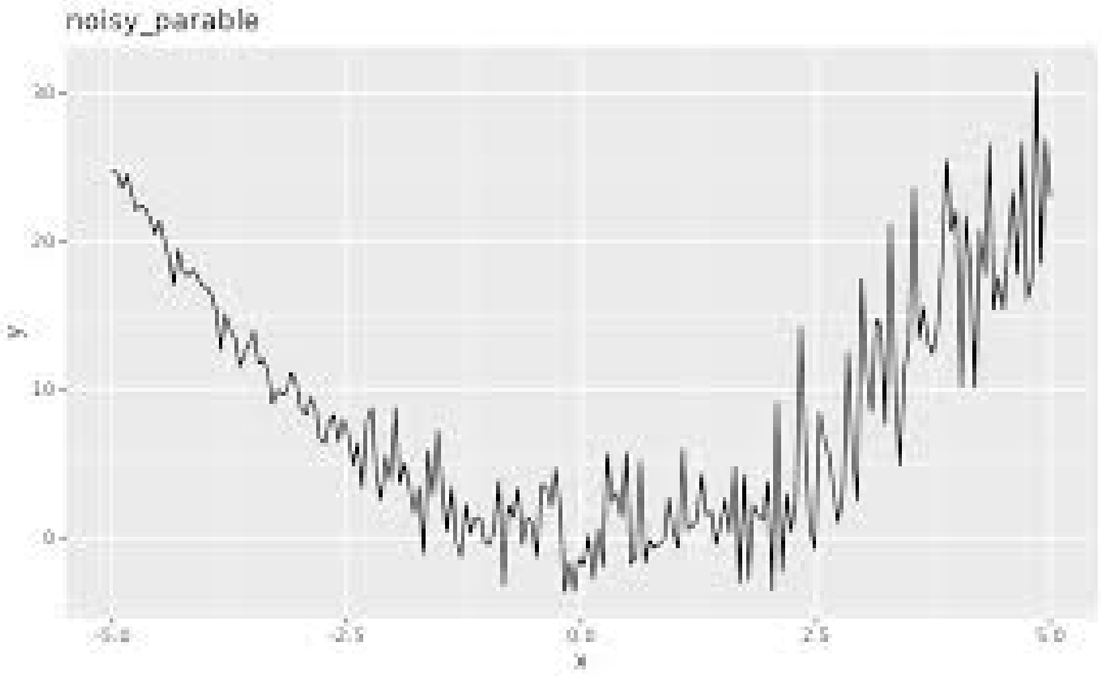
$$\mathbf{q}(c(t))$$



Why use Sampling Based Planners??

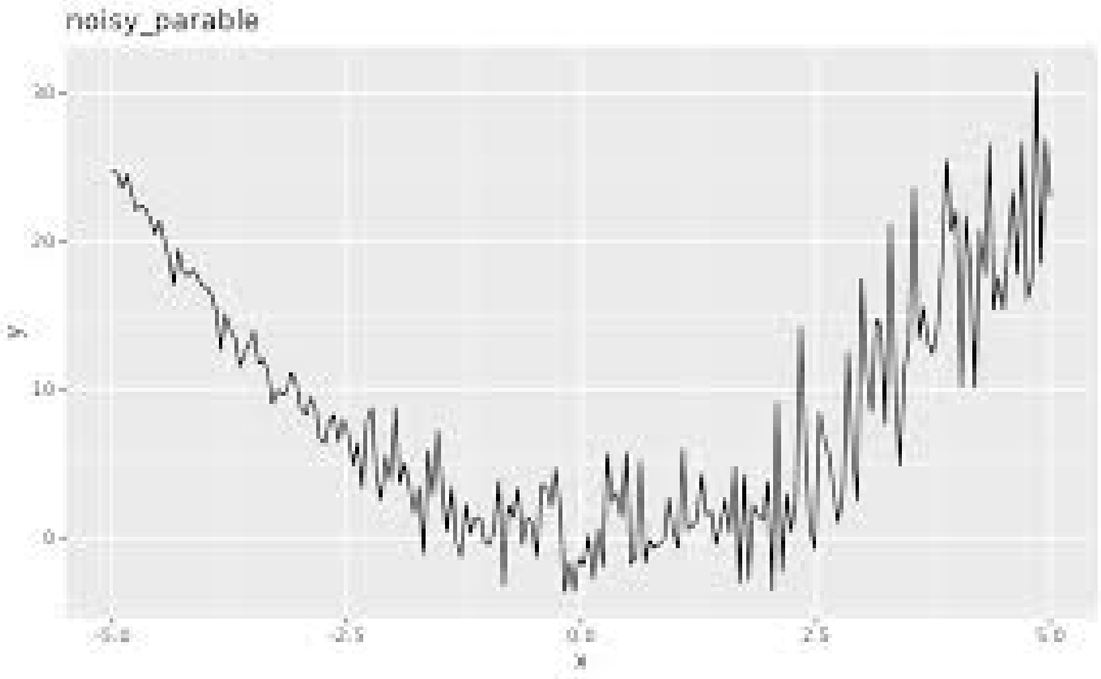
Why use sampling?

- gradient-based methods like CHOMP are powerful but have real limitations, and sampling offers a fundamentally different approach.
- In a noisy gradient, optimizers gets stuck in a local minimum nowhere near the global optimum
- Similarly sampling based planners can work in problems that are non differentiable.



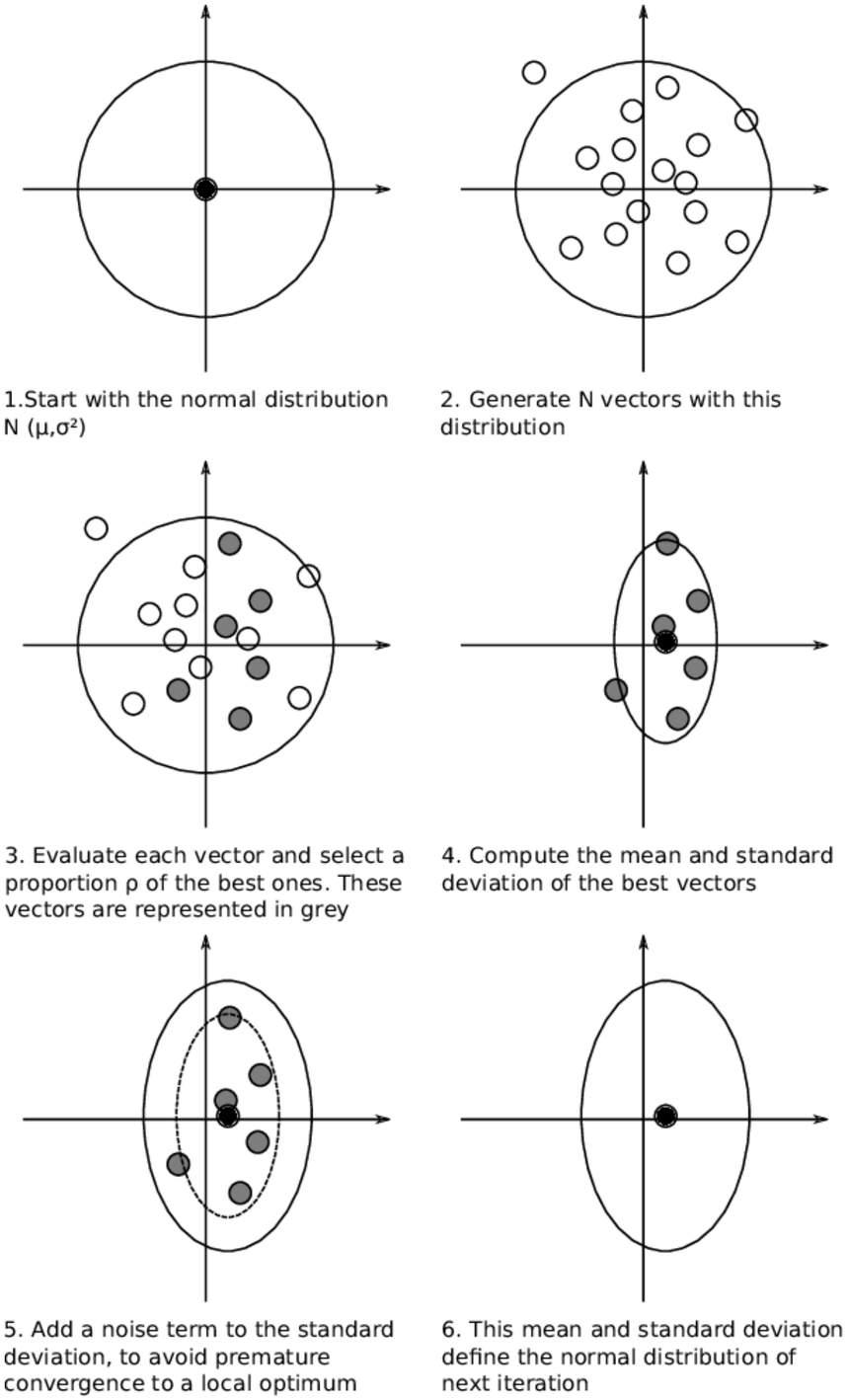
Where to use sampling?

- Sampling is expensive — you need many samples for good coverage. Gradient methods are more efficient when the problem is smooth and well-behaved
- Non convex and problems that require discontinuous or non differentiable cost functions can be easily handled by sampling methods.
- Sampling planners are also highly parallelized, so we can evaluate thousands of trajectories at once.



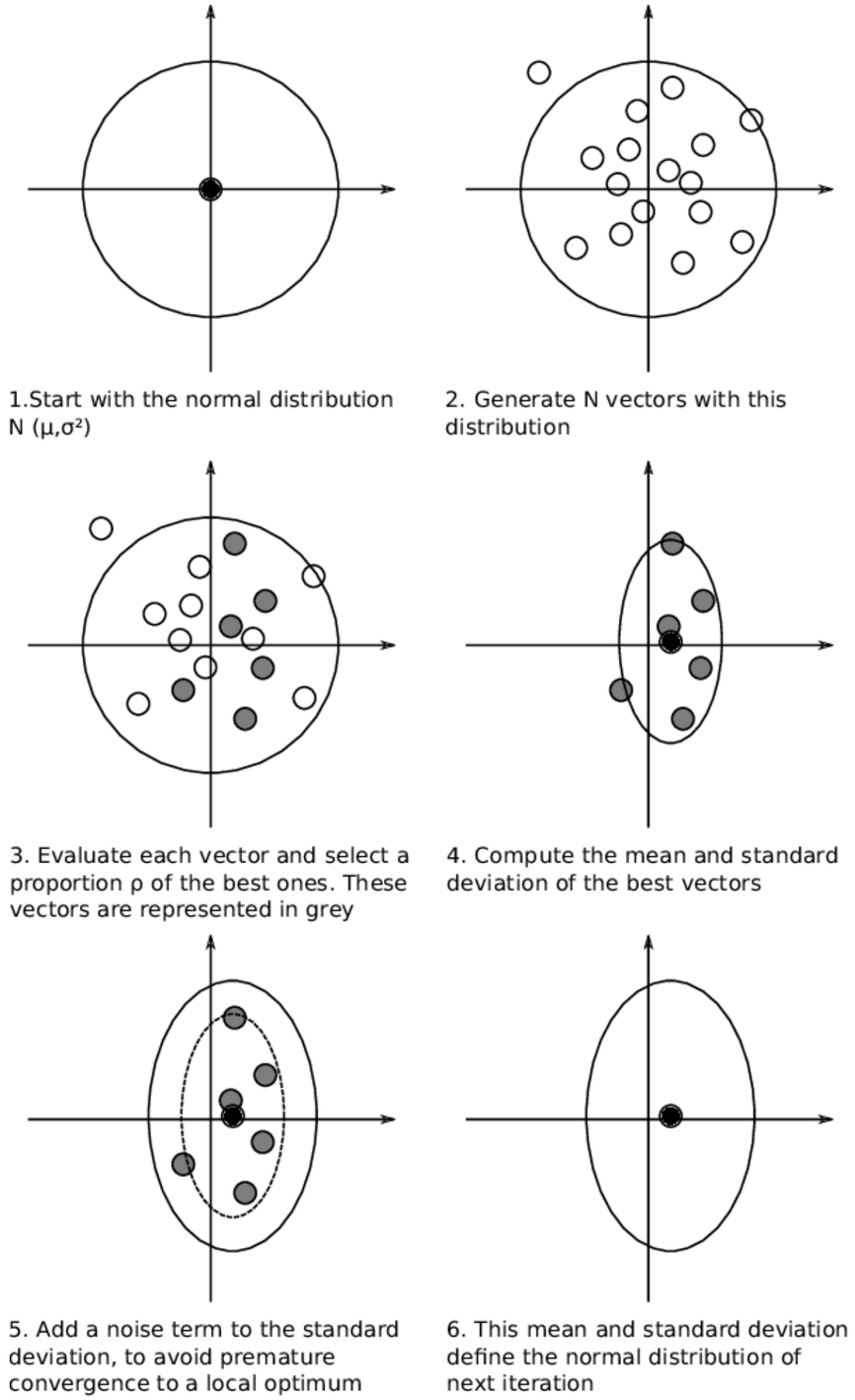
Cross Entropy Method (CEM)

- Sample a large spread of trajectories from a wide Gaussian distribution. Most are bad.
- Evaluate all samples by cost. Get the best “elite” samples.
- fit a new Gaussian to the elite samples only.
- sample again from the new distribution.
- Repeat this till convergence.



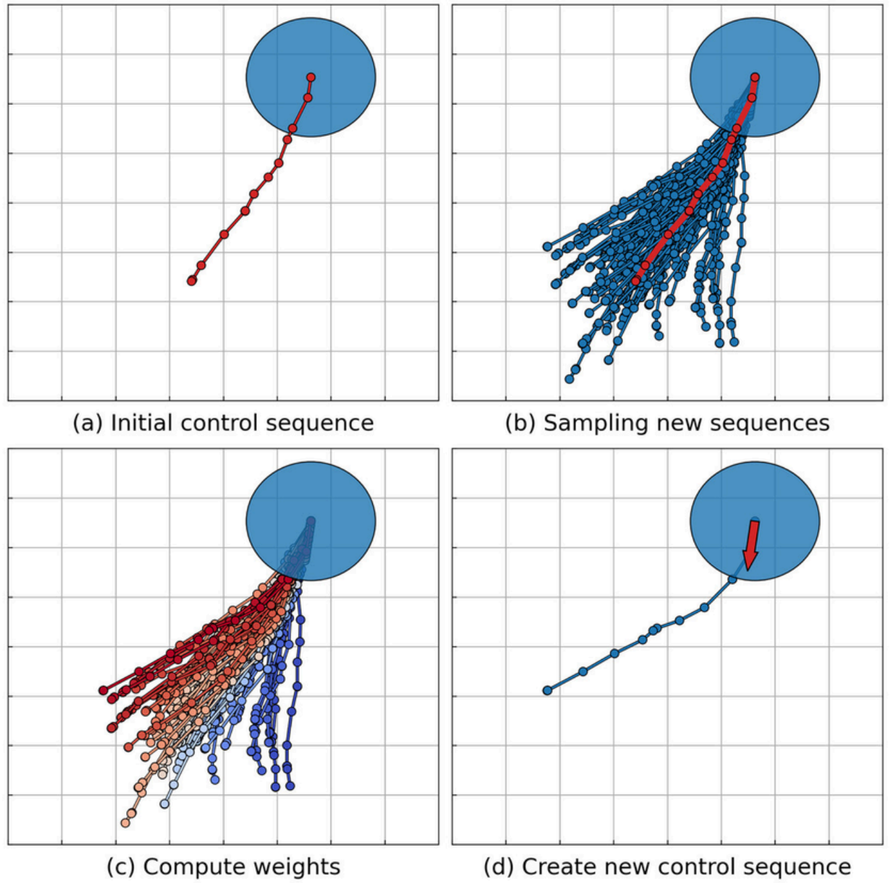
Where to use CEM?

- Good for simple path planning in 2D.
- struggles in very high dimensional spaces
- Very simple to implement.
- need multiple rounds to converge, which makes it slow for real time control.



Model Predictive Path Integral (MPPI)

- A dynamics model — to roll out trajectories forward in time
- A cost function — to evaluate each rollout
- A nominal control sequence — the current best guess, which gets updated each timestep.



Model Predictive Path Integral (MPPI)

$$\mathbf{U} = [\mathbf{u}_0, \mathbf{u}1, \ldots, \mathbf{u}H-1]$$

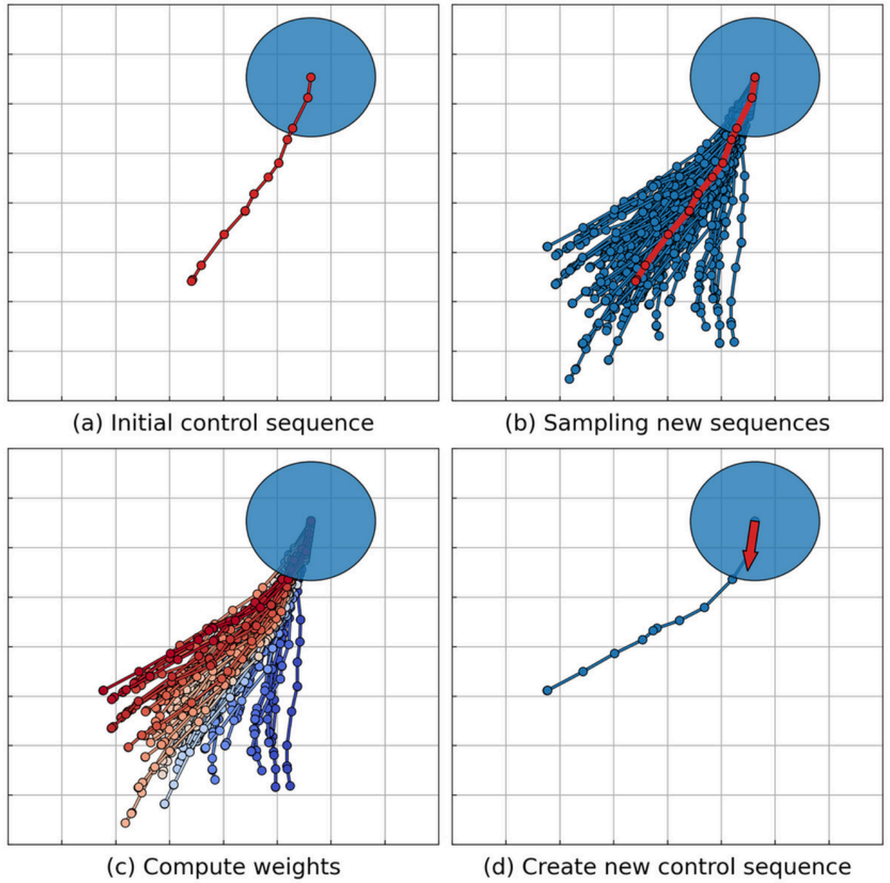
$$\mathbf{U}^i = \mathbf{U} + \boldsymbol{\epsilon}^i, \quad \boldsymbol{\epsilon}^i \sim \mathcal{N}(0, \Sigma)$$

$$\mathbf{x}_{k+1}^i = f(\mathbf{x}_k^i, \mathbf{u}_k + \epsilon_k^i)$$

$$S^i = \sum_{k=0}^{H-1} c(\mathbf{x}_k^i, \mathbf{u}_k^i) + \phi(\mathbf{x}_H^i)$$

$$w^i = \frac{\exp(-\frac{1}{\lambda} S^i)}{\sum_{j=1}^{N} \exp(-\frac{1}{\lambda} S^j)}$$

$$\mathbf{U} \leftarrow \mathbf{U} + \sum_{i=1}^{N} w^i \boldsymbol{\epsilon}^i$$



The End